

VULNERABILITY ANALYSIS PROMPT

Act as a software security analyst. Analyze the code below for potential security vulnerabilities.

For each finding:

1. Identify the vulnerable code and explain the source-to-sink data flow.
2. Describe the conditions required to trigger or exploit the vulnerability.
3. Map the finding to the most specific applicable CWE.
4. Distinguish confirmed evidence from assumptions and missing context.
5. Propose a secure fix and explain why it addresses the root cause.
6. Suggest a test that verifies both the vulnerability and the proposed fix.

Do not invent CVE identifiers, affected versions, execution conditions, or CVSS scores. If the available evidence is insufficient, explicitly state what additional information or testing is required.

CODE AND CONTEXT

Programming language: Python

Application type: Linux command-line utility running with standard user privileges

Attacker-controlled inputs: Data supplied through command-line arguments

Relevant dependencies and versions: None

Code to analyze:

```
def ping():
    cmd = "ping -c 1 %s" % request.args.get("host", "www.google.com")
    status = os.system(cmd) # Noncompliant
    return str(status == 0)
```

OUTPUT

Finding: OS command injection

Severity: Potentially high

CWE: CWE-78 - Improper Neutralization of Special Elements used in an OS Command

Vulnerable code and data flow

```
cmd = "ping -c 1 %s" % request.args.get("host", "www.google.com")
status = os.system(cmd)
```

Data flows as follows:

```
Attacker-controlled host argument
-> string interpolation into cmd
```

```
-> os.system(cmd)
-> execution by the system shell
```

`os.system()` passes the complete string to a shell. Consequently, shell metacharacters in `host` are interpreted as commands rather than as part of a hostname.

For example, a value conceptually equivalent to:

```
127.0.0.1; touch /tmp/ping-injection
```

would produce:

```
ping -c 1 127.0.0.1; touch /tmp/ping-injection
```

The second command would execute with the utility's user privileges.

Exploitation conditions

- The attacker can control the value returned for `host`.
- That value reaches this code without strict validation or escaping.
- `os.system()` invokes an available command shell, as it normally does on Linux.
- The injected command is permitted under the current user's permissions.

Standard-user privileges limit the immediate impact, but an attacker could still read or alter accessible files, execute programs, access user credentials, or otherwise act as that user.

Evidence and missing context

Confirmed from the snippet:

- Untrusted data is concatenated directly into a command string.
- The string is passed to a shell-command execution API.
- There is no visible validation or neutralization.

Assumptions requiring clarification:

- The context says input is supplied through command-line arguments, but the code uses `request.args`, which ordinarily represents HTTP query parameters in frameworks such as Flask. The injection exists under either input model, provided the attacker controls host.
- The surrounding code could theoretically validate `host` before `ping()` is called, but no such validation is shown.
- The exact installed `ping` implementation and executable search path are unspecified.

No CVE, affected version, or CVSS score can be established from this snippet.

Secure fix

Do not invoke a shell. Pass arguments separately and validate that the target is a hostname or IP address:

```
import ipaddress
import re
import subprocess
_HOSTNAME_RE = re.compile(
    r"(?=[1,253]\Z)"
    r"(?:[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?\.\.)*"
    r"[A-Za-z0-9](?:[A-Za-z0-9-]{0,61}[A-Za-z0-9])?"
```

```

r"\Z"
)
def valid_target(value):
try:
ipaddress.ip_address(value)
return True
except ValueError:
return bool(_HOSTNAME_RE.fullmatch(value))
def ping():
host = request.args.get("host", "www.google.com")
if not valid_target(host):
return "invalid host"
result = subprocess.run(
["/usr/bin/ping", "-c", "1", host],
stdin=subprocess.DEVNULL,
stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL,
timeout=10,
check=False,
)
return str(result.returncode == 0)

```

This addresses the root cause because:

- `subprocess.run()` receives an argument list and does not invoke a shell.
- Shell characters cannot become command separators.
- Validation rejects command syntax and option-like inputs such as `-f`.
- An absolute executable path avoids executing an attacker-controlled ping found through `PATH`.
- A timeout prevents the process from hanging indefinitely.

If arbitrary non-host targets must be supported, the argument-list form remains essential, but the validation policy should be adjusted deliberately.

Verification tests

A regression test for the original implementation should supply a payload that creates a uniquely named temporary file:

```

def test_original_is_command_injectable(client, tmp_path):
marker = tmp_path / "injected"
payload = f"127.0.0.1; touch {marker}"
client.get("/ping", query_string={"host": payload})
assert marker.exists()

```

Run this only in an isolated test environment. It demonstrates the vulnerability if the marker appears.

Tests for the fixed implementation should verify both rejection and safe process construction:

```

def test_rejects_shell_payload(client, tmp_path):
marker = tmp_path / "injected"
payload = f"127.0.0.1; touch {marker}"
response = client.get(
"/ping", query_string={"host": payload}
)
assert response.data == b"invalid host"
assert not marker.exists()
def test_uses_argument_list(monkeypatch, client):

```

```
captured = {}
def fake_run(args, **kwargs):
    captured["args"] = args
    class Result:
        returncode = 0
    return Result()
monkeypatch.setattr(subprocess, "run", fake_run)
response = client.get(
    "/ping", query_string={"host": "127.0.0.1"}
)
assert captured["args"] == [
    "/usr/bin/ping", "-c", "1", "127.0.0.1"
]
assert response.data == b"True"
```

Additional tests should cover valid IPv4, IPv6, DNS names, leading-dash inputs, invalid labels, excessive length, subprocess timeout, and systems where `/usr/bin/ping` is absent.